

Sauron

O Sauron organiza a fila e a alocação de GPUs do cluster. Você leva seu código para a máquina, descreve o job num arquivo, e o Sauron roda seu container numa GPU livre e avisa quando termina.

Acesso

Entre por SSH com o seu usuário, no endereço que o administrador te enviou. A senha é a mesma que você já usa nas outras máquinas do laboratório:

```
ssh <seu-usuario>@<endereço-do-login>
```

Ao conectar você cai na sua sessão de login.

A sessão de login

A sessão de login serve para preparar código, editar arquivos, enviar jobs e acompanhar resultados. Ela não tem GPU nem acesso a Docker ou `nvidia-smi`.

Atenção

Não rode treinos, inferências, notebooks ou processos pesados na sessão de login. Todo processamento roda nas GPUs pelo `sauron submit`.

Levar o código

Na sessão de login, clone seu projeto numa subpasta de `~/workspace`, o disco rápido que o job enxerga. Não deixe código, dados ou modelos direto na home (`~`): ela é pequena e lenta.

```
cd ~/workspace
git clone <url-do-seu-repo> meu-projeto
```

Escrever o job.yaml

Gere um `job.yaml` respondendo algumas perguntas:

```
sauron new
```

Um `job.yaml` típico:

```
name: meu-treino
image: pytorch/pytorch:2.13.0-cuda13.0-cudnn9-devel
gpus: 1
vram: 80                      # VRAM mínima por GPU, em GB
max_time: 8h
command: python train.py

workspace: meu-projeto        # subpasta de ~/workspace; o job já começa dentro dela
```

Campos opcionais: `cpus`, `memory` (ex.: `8g`), `env` (só variáveis *não* secretas) e `volumes` (montar dados extras). Uma chave escrita errada gera erro no envio, então não passa em branco.

`vram` escolhe a classe de GPU pelo mínimo de VRAM, não uma fração. As classes disponíveis aparecem ao vivo no `sauro new` e no `sauro queue`: peça a menor que te atende.

Vários jobs rodam na mesma máquina ao mesmo tempo e dividem a mesma RAM. `memory` é o teto de RAM do seu container. Se o job passar do teto, só ele é encerrado por falta de memória, não a máquina. Declare um valor realista com `memory:` (ex.: `16g`): pedir demais tira memória de quem também precisa, pedir de menos derruba o seu job.

Imagem e dependências

O campo `image` diz em qual container o seu job roda. O Sauron baixa essa imagem pronta de um registry, como o Docker Hub ou o NGC da NVIDIA, e roda o seu comando dentro dela. Você não precisa de Dockerfile. Na primeira vez em que uma imagem é usada, a máquina baixa a imagem antes de começar o job; acompanhe o download pelo `sauro logs`.

Caminho recomendado: imagem pronta + suas dependências no script

Escolha uma imagem que já traga CUDA e o framework que você usa:

```
image: pytorch/pytorch:2.13.0-cuda13.0-cudnn9-devel
```

Instale o resto num venv, no início do job, pelo `script.sauro`:

```
set -e
export PYTHONNOUSERSITE=1
python -m venv --without-pip --system-site-packages .venv
. .venv/bin/activate
python -m pip install -r requirements.txt
python train.py
```

Envie com `sauron submit -c job.yaml script.sauron`. O cache do pip fica guardado entre um job e outro, então a partir da segunda vez a instalação fica mais rápida.

O venv fica na pasta do projeto, aproveita os pacotes que já vêm na imagem e vale para os próximos jobs. Para começar do zero, apague a pasta `.venv`.

O `export PYTHONNOUSERSITE=1` impede que pacotes antigos de `~/workspace/.local` tomem o lugar dos pacotes do venv.

Imagem própria

Se o seu ambiente é grande ou demora para instalar, vale montar uma imagem própria a partir de uma das imagens acima:

1. Construa a imagem no seu computador ou em outra máquina com Docker.
2. Publique num registry público, como Docker Hub ou GitHub Container Registry.
3. Use o nome publicado no `job.yaml`, com uma versão fixa:

```
image: seu-usuario/meu-ambiente:v3
```

Use sempre uma versão fixa (`v1`, `v2`, `2026-09-25`) e troque o número a cada mudança. A máquina guarda a imagem depois do primeiro uso. Com um nome novo, você garante que o job vai rodar a versão que acabou de publicar.

Enviar e acompanhar

<code>sauron submit -f job.yaml</code>	envia o job
<code>sauron list</code>	lista seus jobs mais recentes
<code>sauron status</code>	tempo de espera por tamanho de GPU
<code>sauron status <ref></code>	estado do job e posição na fila
<code>sauron logs <ref></code>	mostra os logs
<code>sauron logs <ref> -f</code>	acompanha os logs em tempo real, começando pelas últimas 100 linhas
<code>sauron logs <ref> -n N</code>	mostra só as últimas N linhas (<code>-n all</code> para tudo); combina com <code>-f</code>
<code>sauron queue</code>	mostra a fila e a estimativa de espera

A referência `<ref>` é o número do job, mostrado no `list`. Para controlar um job:

<code>sauron cancel <ref></code>	encerra dando tempo para finalizar
<code>sauron kill <ref></code>	encerra na hora
<code>sauron retry <ref></code>	recoloca na fila um job que falhou

Para vários comandos, com `source`, pipes ou `&&`, coloque o corpo num arquivo `.sauron` e use um `job.yaml` sem a linha `command`:

```
sauron submit -c job.yaml script.sauron
```

vLLM

Para rodar modelos de linguagem com o vLLM, use a imagem oficial `vllm/vllm-openai`. Ela já traz o vLLM, o CUDA e o pacote `openai`. Baixe o `vllm.yaml` e o `vllm.sauron` para a pasta do projeto e envie com:

```
sauron submit -c vllm.yaml vllm.sauron
```

Sempre com -c

Com a imagem do vLLM, envie o job com `-c` e um arquivo `.sauron`, mesmo que seja um comando só.

Gerar em lote

Para gerar respostas para muitos prompts de uma vez, chame o vLLM direto do seu código Python:

```
from vllm import LLM, SamplingParams

llm = LLM("Qwen/Qwen2.5-7B-Instruct")
prompts = ["Explique o que é uma GPU em uma frase.", "Dê um nome para um gato."]
saidas = llm.generate(prompts, SamplingParams(max_tokens=100))
for saida in saidas:
    print(saida.outputs[0].text)
```

Nesse caso o `vllm.sauron` tem uma linha só: `python3 gerar.py`. Na imagem do vLLM o comando é `python3`.

Servidor e o seu código no mesmo job

Se o seu código conversa com uma API no formato da OpenAI, suba o servidor do vLLM no próprio job. O `vllm.sauron` do kit faz isso: sobe o servidor, espera o modelo carregar e roda o seu código. Se o servidor cair, o job mostra o motivo e encerra.

```
set -e
```

```
MODEL=Qwen/Qwen2.5-7B-Instruct
```

```
vllm serve "$MODEL" --port 8000 > vllm.log 2>&1 &
```

```
servidor=$!
```

```
trap 'kill $servidor 2>/dev/null' EXIT
```

```
until curl -sf localhost:8000/health > /dev/null; do
```

```
    kill -0 $servidor 2>/dev/null || { tail -50 vllm.log; exit 1; }
```

```
    sleep 5
```

```
done
```

```
python3 cliente.py
```

No seu código, aponte o cliente para `localhost:8000` :

```
from openai import OpenAI
```

```
cliente = OpenAI(base_url="http://localhost:8000/v1", api_key="local")
```

```
resposta = cliente.chat.completions.create(
```

```
    model="Qwen/Qwen2.5-7B-Instruct",
```

```
    messages=[{"role": "user", "content": "Explique o que é uma GPU em uma frase."}]
```

```
)
```

```
print(resposta.choices[0].message.content)
```

As mensagens do servidor ficam em `vllm.log` , na pasta do projeto.

Segredos

Segredos só no `.env`

Segredos nunca vão no `job.yaml` nem no comando, o envio recusa tokens e chaves. Grave um arquivo `.env` na pasta do projeto e carregue dentro do job:

```
source .env
```

Notificações no Telegram

Receba aviso quando o job começa, termina ou falha:

```
sauron telegram connect
```

Escaneie o QR que aparece no terminal (link válido por 15 min) e toque em START. O aviso de falha indica como ver o log. Confira o estado com `sauron telegram show`.

Com o chat vinculado, mande `/status` pro bot para ver a disponibilidade do cluster na hora: o tempo de espera por tamanho de GPU. Com `/status 42` você vê o estado de um job seu, e `/jobs` lista seus jobs ativos e os últimos terminados.

Mande `/logs` para ver o final do log do seu job (`/logs 42` escolhe o job). Com `/autologs` o log passa a vir dentro do próprio aviso de falha; mande de novo para desligar.

Mandar feedback

Achou um problema ou tem uma sugestão? Fale com os mantenedores direto do terminal:

```
sauron feedback "sua mensagem"
```

Armazenamento e backup

Sua pasta na máquina é espaço de trabalho, não repositório permanente. Limpe com regularidade o que não usa mais: caches, arquivos temporários, checkpoints antigos e imagens de container.

Sem backup

O cluster não faz backup automático, e uma falha de hardware pode levar tudo. Mantenha fora do cluster a cópia principal de código, datasets, checkpoints e resultados importantes.

Boas práticas

- Antes de um treino longo, envie uma versão curta (poucos passos, `max_time` de minutos). Erro de caminho, dependência ou memória aparece em minutos, não depois de horas na fila.
- Peça a menor classe de `vram` que te atende. Classe menor costuma liberar antes, e o uso recente de GPU pesa no desempate da fila.

- Dê folga ao `max_time` , mas sem exagero: o job é encerrado ao passar do limite, e o valor entra na estimativa de espera de todo mundo.
- Salve checkpoints periodicamente. Se o job falhar ou estourar o tempo, o `sauron retry` recomeça do zero a não ser que seu código retome do último checkpoint.

Ajuda

`sauron guide`

mostra o guia rápido no terminal

`sauron --help`

lista todos os comandos

`sauron <comando> --help`

ajuda de um comando

Atualizado em 03/10/2026